

Receiver-Guided Reinforcement Learning-based Congestion Control for IoT network

Qihang Jiao, *Student Member, IEEE*, and Lotfi Mhamdi, *Senior Member, IEEE*

Abstract—Internet of Things (IoT) have been widely deployed and remains a growing trend in the globe. Connection techniques are improved leading to the massive IoT connected to the network and generating numerous data. Most of IoT devices send data following client-server manner where massive data are processed centrally. For reliability sensitive applications e.g., autonomous mobile robot control and quality assurance require TCP accompanied with congestion control (CC). In particular, the manner in which IoT devices generate traffic can be categorised into periodic and aperiodic according to different application scenarios. Existing CC algorithms have been widely researched by approaching different congestion indicator e.g., loss, delay, and bandwidth delay product (BDP). Loss-based CCs are the most favoured whilst they are pointed out exacerbating the queueing in the intermediate node. Other CCs are still under estimations before deployment. Reliable CC researches that able to alleviate congestion for massive flows are accomplished in data centre considering different characteristics from IoT scenarios, e.g., base RTT and traffic characteristics. Reinforcement Learning (RL) has been widely explored and considered able to generate better CC for most cases. However, existing CC researches are not sufficiently evaluated considering IoT networks. We evaluated existing CC schemes in IoT networks considering IoT traffics and implemented a receiver-guided congestion control (RGRL) using Reinforcement Learning.

Index Terms—IoT, TCP, Congestion Control, Reinforcement Learning.

I. INTRODUCTION

Internet of Things (IoTs) have been widely deployed with the improving advantages of connection techniques. Ubiquitous sensing and computing capabilities provided by heterogeneous physical objects motivates the emergence of applications requiring varied data. The applications include transportation, industrial automation, emergency response. At the present when computation capability is highly demanded and increased, IoT is inevitably becoming the integral part of the future Internet. However, with the heterogeneous physical objects connected to the Internet, massive data are generated and transmitted in to network leading to severe network congestion. The network congestion from massive devices introduces the challenge in terms of reliability, latency, and fairness which are critical for applications requiring a high quality of service (QoS). With the expending device connections, network lifespan, connection quality, control overhead, and the heterogeneity are also introduced challenge to traditional

technology. An IoT device is usually considered resource-constrained that leads to energy consumption as an important factor affecting the network scalability [1], [2], [3].

For the energy efficient and reliable data transmission, congestion control (CC) over lightweight UDP-based protocol CoAP has been proposed [4]. However, UDP is suspected with its network safety problem and managed strictly by corporate firewalls. Therefore, UDP-based protocols cannot satisfy cross domain scenarios. For scenarios demanding high reliability, high proportion of TCP usage is observed in the recent investigation especially for autonomous mobile robot along with quality assurance in production [5], [6]. Those requirements to network performance in traditional network that also applied to IoT, remains the diversity in choosing transport protocol [7], [8], [9]. As IoT network is proposed to be a component to the future Internet, existing CC performance needs to be further researched in IoT.

Conventional CCs relies on loss as congestion indicator, e.g., NewReno [10] and Cubic [11]. They are mainly used for TCP/IP stack devices and has proven their reliable capability after decades. They are not optimal in multiple senders scenario in terms of RTT and fairness, as they rely on greedy increment to *cwnd* to probe the available bandwidth. Therefore, a buffer in the network must be overflowed frequently to indicate a congestion occurrence for senders to multiplicative decrease their *cwnd*. This ensure the availability of bandwidth at the cost of queue build up, resulting a high RTT and different share of bandwidth among the traffics. Latency-based CCs, e.g., [12] are designed to measure a minimal RTT while they face the challenge of obtaining the global minimal RTT when the network is congested by the greedy loss-based CCs. A hybrid CC, BBR [13] is being highly focused which periodically probes the maximum available bandwidth and the minimal RTT to measure the bandwidth delay product (BDP).

Abundant CC schemes optimised for large scale of traffics are accomplished in data centre networks, where the base RTT is much lower than IoT networks, e.g., hundreds of microseconds [14]. In DCTCP the researchers utilise explicit congestion notification (ECN) marking to make the TCP sender response to the congestion with a adaptive extent to achieve a better fairness and low latency for senders of larger scales. DCTCP accomplished a lower flow completion time compared to traditional TCP. However, DCTCP is specified for data centre where the base RTT is only several microseconds indicating a very rapid congestion control loop. The researchers mentioned DCTCP is not claimed to apply to wider network like IoT networks. In SAB [15], the researchers achieve a better fairness with lower queueing delay in the circumstance that

Qihang Jiao and Lotfi Mhamdi are with the School of Electronic and Electrical Engineering, University of Leeds, Leeds, UK (e-mail: el22qj@leeds.ac.uk; L.Mhamdi@leeds.ac.uk).

Lotfi Mhamdi is also with the College of Computing & Systems, Abdullah Al Salem University, Kuwait (e-mail: lotfi.mhamdi@asu.edu.kw).

when bottleneck buffer size is larger bandwidth delay product (BDP). However, their proposal requires massive modifications to the intermediate nodes. For a service through the internet, the intermediate nodes that forms a backlog is hard to obtain, along with the requirement of the suitable buffer size should be determined according to the BDP in the network.

Reinforcement Learning (RL) is considered a potential solution to the dynamic environment by learning a policy to interact with the environment. QTCP [16] leverages Q-Learning to control sender *cwnd* relying on the difference between a defined utility function value of the latest two steps. Aurora [17] is described as a framework extended from PCC [18], where the researchers introduce neural network in their RL design aim to make CC adapt to varying network scenarios. Fairness, multiple objectives, and adaptation to dynamic network conditions remain the key challenges in RL-based CCs. Orca [19] is trained in a hybrid scheme that employs both advanced DRL and classic CCs. They demonstrate the RL-based schemes challenges are converge to bandwidth, performances issues over unseen conditions, and computation overhead. It is observed in [20] that in most scenarios unfairness problem still exist among the existing RL-based CCs. The researchers also point out that existing RL-based CCs focusing on capturing available bandwidth swiftly but introducing severe packet losses in normal operation.

In this work, we design our RL-based scheme in the principles of minimal deployment and controlling the queueing delay for better scalability to suit the resource-constrained and massive amount devices in IoT networks. We propose Receiver-guided RL-based (RGRL) CC for IoT. RGRL works on top of additive increase multiplicative (AIMD) decrease mechanism CC e.g., NewReno to estimate the network state from the receiver to adjust sender *cwnd* by exchanging information in modified TCP header.

II. RELATED WORKS

A. Reliable Communications and Data Traffic in IoT

Data traffic in IoT scenario has been investigated in industry [5]. The researchers investigate packet size and packet inter-arrival time distribution from three different cases, unit test cell, visual inspection cell, and autonomous mobile robot. It is observed that TCP is mainly used in autonomous mobile robot, particularly, in fleet manager mode and manual controller mode. Within the scope of 5G IIoT, they mentioned typical traffics used for standard research are categorised into periodic and aperiodic.

A more detailed analysis is conducted in [21]. The researchers specify their considered IoT scenario as a client-server manner where a large number of devices collecting data to be aggregated centrally. Scalability is turn to a challenge in this case, especially when the number of devices grows exponentially in an architecture where brokers and gateways can be placed dynamically. They analysed the this aggregated traffics as asynchronous periodic traffic and compared it with standard poissonian traffic. They mixed asynchronous periodic traffic with 10% poissonian traffic and varied the nodes number generating periodic traffic. It is observed that the

cumulative distribution function of the mixed traffic is far different from the pure poissonian traffic when the number of nodes is low, but perform similar when the node number is higher.

B. Reinforcement Learning-based Congestion Control

Reinforcement Learning (RL) involves agents learning to interact with an environment to maximise a long-term objective. With the use of function approximator e.g., neural network and numerous experience from extensive training, RL has potential to learn an algorithm better than pre-defined rules for many applications. RL-based CCs are researched out of the prospect to extend RL capabilities into optimising network performances.

QTCP [16] leverages Q-learning to directly change the *cwnd*. They use their own defined utility function and assign fixed reward according to the change of the utility value. In PCC [18], the researchers introduced online-learning to rate control to solve the problem that the conventional CCs performance impaired. They attribute this impairment to human defined model failed to reflect the complexity of networks. Aurora [17] modeled the process using deep RL method, particularly, PPO to conduct their CC to Internet. TCP-RL [22] is implemented using RL to dynamically configure initial window and CC algorithm as they formulate the problem of CC that unnecessary time experienced for growing *cwnd* and performance varies in different network conditions. Orca [19] demonstrate a RL-based framework where a monitor block cooperate with an agent to output a change to *cwnd* to underlying TCP CC periodically. Their scheme is capable to compatible to any existing CC. SPINE [23] improves the control granularity and lower the computation overhead by introducing hierarchical design. Sage [24] introduces offline RL with sufficient training to generalise the performance improvement for practicality in Internet. RL-based CC remain imperfect performance in terms of fairness, efficiency, and generalisation [20].

Few works discussed the TCP performance in the scope of IoT network where numerous devices generating massive data. In this work, we discuss existing TCP performance in IoT scenario and propose our work RGRL with a RL framework to improve the scalability of CC for IoT networks. We consider practicality in our design by introducing receiver-guided scheme and tested with IoT specified traffic model.

III. IIoT TRAFFIC MODEL

For evaluating existing CCs performance in IoT scenario and explore their scalability, specific traffic model is required. We considered the common scenarios in CC research client-server scenario as in [21], where a large number of devices generating data to be aggregated centrally. A common traffic pattern is periodic traffic generated using Equation. 1, where $s_{i,k}$ denotes the moment when a message is generated by device i in period k .

$$s_{i,k} = t_i + kT_i, \quad k \in \mathbb{N} \quad (1)$$

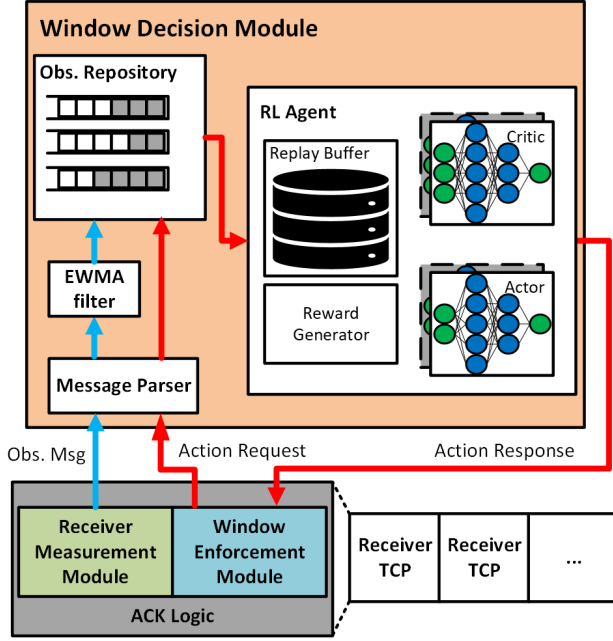


Fig. 1: System architecture of our proposal.

Event-based IoT application is also usually used. For simulating this scenario, they behave as aperiodic traffic and are approximated using Poisson process. Therefore, aperiodic traffics are simulated by sampling their inter-arrival time (Δs) according to Equation. 2.

$$\Delta s \sim \text{Exp}(\lambda) \quad (2)$$

Under the definitions of 5G, [25] summarises traffic characteristics comprehensively of 3GPP definitions. Various parameters are included in [25] including the bytes in a session, which we refer to as message size in our work. It is mentioned for smart metering, point of sale, and home automation, a message is small, i.e., having only several bytes. However, for fleet management, telemedicine, a message are in the order of KBs. For environment monitoring and control, it is mentioned that a message can have several MBs, which triggers us to consider the flow completion time considering the message in the order of KBs. We emphasise the importance of minimising the tail flow completion time (FCT) that lower tail FCT can benefit the information from devices to be delivered on time enhancing the performance of IoT application. Considering a long flow completion time, the data collected from the devices may not reflect the actual state of the device and causes information lags for IoT applications. Therefore, for supporting larger data than few KBs for IoT applications, we consider CCs performance under larger messages. The earliest definitions of traffic characteristics according to applications can be found in [26].

IV. SYSTEM DESIGN

Our proposal is designed to be receiver-guided, and not blocking normal TCP behaviour. Therefore, we extended and modified the standard TCP to exchange message between

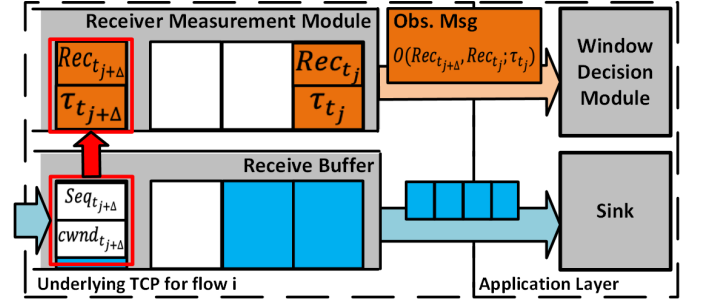


Fig. 2: An example of estimation task completion of flow i .

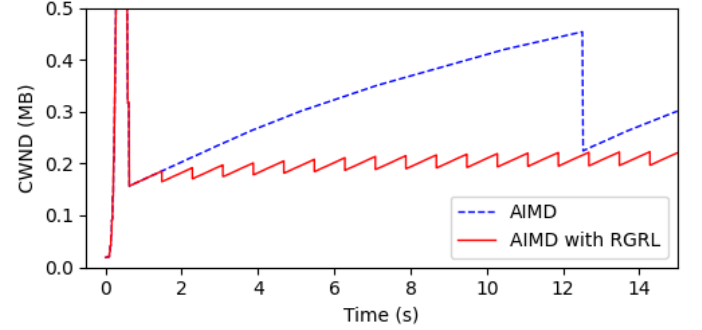


Fig. 3: CWND behaviour.

receiver and sender, as well as between the receiver and agent. We introduce our control message design and the information exchange mechanism to enable receiver-guided congestion control for TCP. Then we present three modules in the receiver side for adjusting sender $cwnd$ relying on the receiver-guided method i.e., Receiver Measurement Module, Window Decision Module, and Window Enforcement Module.

A. Enable Receiver-Guided for TCP

1) *Control Message*: The receiver TCP socket is extended to support communication with the RL agent. In our design, generating an observation and an action are decoupled to ensure the packet communication will not be blocked by the agent inference. We define three types of control messages as following:

- **Observation Message**: carries the observed network state and is stored in the replay buffer.
- **Action Request**: requests the agent to infer and return a control action.
- **Connection Close**: informs the agent that a flow has terminated so that the corresponding internal state can be cleared.

2) *Exchange information through Modified TCP*: We utilise the window size field in TCP header to exchange the information required both by the receiver and the sender. In particular, the sender TCP is modified to include the current $cwnd$ value in the window size field of TCP header. For an enforced $cwnd$, when the receiver TCP receives a control signal from the Window Decision Module, it sets its state as control start, and keeps including the enforced $cwnd$ value into the window size field of ACK with **ECE**. When the sender receives an **ECE**

ACK, it enters a window adjustment phase and overwrites its current *cwnd* to the enforced value. During this phase, replicative ACKs with **ECE** are ignored by the sender so that the enforced value only take effect once in a same phase. Once the window adjustment is completed, the sender TCP will keep sending **CWR** packet and exit the window adjustment phase. Then, the receiver will reply with a **CWR** and both sender and receiver will exit the control phase. By applying the mechanism above, our design can achieve a value exchange reliably when encountering packet loss. The *cwnd* behaviour of RGRL cooperating with AIMD can be seen in Fig. 3.

B. Receiver Measurement Module

Our scheme relies on reliable state observations to ensure stable training. Therefore, the states in our design are required to reflect the congestion extent while remaining observable at the receiver. Despite RTT is a fine-grained and valuable indicator of congestion, it is difficult to directly observe in the receiver side.

To address the limitation above, we derive a receiver-side congestion extent metric based on the *cwnd* elapsed time ($t_{elapsed}$). We extend the sender TCP feature to include its *cwnd* in TCP header. In standard TCP congestion avoidance, the congestion window *cwnd* is increased by 1 MSS after an entire *cwnd* bytes has been acknowledged. This nature results in an approximate growth rate of 1 MSS per RTT, known as additive increase.

We utilise additive increase growing feature to enable the receiver to estimate network congestion extent by calculating the elapsed time for a flow to complete its *cwnd*. In particular, upon every packet receive event that includes a *cwnd* value different from the last value received, the Receiver Measurement Module establishes a new estimation task.

An example of observation message generation can be seen in Fig. 2. A new task is initialised with a record (*Rec*) for the current flow state and a threshold (τ), e.g., for flow *i* started at time t_0 and estimated at time t_j , defined as below:

$$Rec_{i,t_j} \triangleq [B_i(t_j), L_i(t_j), t_j] \quad (3)$$

where we set τ to be the sum of sequence number (Seq_{i,t_j}) and the *cwnd* value from its header. We also maintain a cumulative received bytes ($RxBytes_{i,t}$) and cumulative packet losses ($Losses_{i,t}$) counter in the receiver TCP, defined as follows:

$$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j} \quad (4)$$

$$B_i(t_j) = \sum_{n=t_0}^{t_j} RxBytes_{i,n} \quad (5)$$

$$L_i(t_j) = \sum_{n=t_0}^{t_j} Losses_{i,n} \quad (6)$$

An estimation task is completed at $t_{j+\Delta}$ when a coming packet is seen with a sequence number satisfying $Seq_{i,t_{j+\Delta}} > \tau_{i,t_j}$. We calculate this task duration as the *cwnd* elapsed time i.e., $t_{elapsed} = t_{j+\Delta} - t_j$. We then calculate the goodput (g_{i,t_j}) and packet losses during the estimation task, where $g_{i,t_j} = \frac{B_i(t_{j+\Delta}) - B_i(t_j)}{t_{elapsed}}$. and $\ell_{i,t_j} = L_i(t_{j+\Delta}) - L_i(t_j)$.

Algorithm 1 Receiver Measurement and Observation Generation

Input: Flow Id *i*, Time t_j , Arrived Packet p_{i,t_j} , Flow Observation Queue q_i

Output: Observation o_{i,t_j} , Control Message *M*

- 1: Extract Seq_{i,t_j} , $cwnd_{i,t_j}$, and $RxBytes_{i,t_j}$ from p_{i,t_j}
 - 2: $\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j}$
 - 3: Obtain $Losses_{i,t_j}$ from *TcpSocket_i*
 - 4: $B_i(t_j) = B_i(t_{j-1}) + RxBytes_{i,t_j}$
 - 5: $L_i(t_j) = L_i(t_{j-1}) + Losses_{i,t_j}$
 - 6: $Rec_{i,t_j} = [B_i(t_j), L_i(t_j), t_j]$
 - 7: Obtain Rec_{i,t_j}^{head} and τ_{i,t_j}^{head} from the head of q_i
 - 8: **if** $Seq_{i,t_j} \geq \tau_{i,t_j}^{head}$ **then**
 - 9: Dequeue Rec_{i,t_j}^{head} and τ_{i,t_j}^{head} from q_i
 - 10: $o_{i,t_j} = O(Rec_{i,t_j}, Rec_{i,t_j}^{head})$
 - 11: Include o_{i,t_j} into *M*
 - 12: Send *M* to agent
 - 13: **else if** $cwnd_{i,t_j} \neq cwnd_{i,t_{j-1}}$ **then**
 - 14: Enqueue Rec_{i,t_j} and τ_{i,t_j} into q_i
 - 15: **end if**
-

Algorithm 2 Window Decision Module

Input: Control Message *M*, Replay Buffer *B*, Flow State Buffer B_i for flow *i*, Batch Size *N*

Output:

- 1: **if** *M* = "Connnection Close" **then**
 - 2: Clear B_i
 - 3: **else if** *M* = "State" **then**
 - 4: Extract s_i from *M*
 - 5: Store s_i into B_i
 - 6: **else if** *M* = "Action Request" **then**
 - 7: Compute the average state $\bar{s}_i$ over B_i
 - 8: $a = \mu(\bar{s}_i; \theta^\mu)$
 - 9: Store transition $(\bar{s}_i, a_i, r_i, \bar{s}_i')$ into *B*
 - 10: Randomly sample *N* transition from *B*
 - 11: Compute $y_t = r(s_t, a_t) + \gamma Q^-(s_{t+1}, \mu^-(s_{t+1}))$
 - 12: Compute $L(\theta^Q) = \mathbb{E}[(Q(s_t, a_t; \theta^Q) - y_t)^2]$
 - 13: Update Critic minimising $L(\theta^Q)$
 - 14: Compute $\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_{a_t} Q(s_t, \mu(s_t)) \nabla_{\theta^\mu} \mu(s_t)]$
 - 15: Update Actor via gradient ascent $\nabla_{\theta^\mu} J(\theta^\mu)$
 - 16: $\theta^{\mu^-} \leftarrow \tau \times \theta + (1 - \tau) \times \theta^{\mu^-}$
 - 17: $\theta^{Q^-} \leftarrow \tau \times \theta + (1 - \tau) \times \theta^{Q^-}$
 - 18: **end if**
-

Therefore, an observation is formed denoted as $o_{i,t_j} = O(Rec_{i,t_{j+\Delta}}, Rec_{i,t_j}; \tau_{i,t_j})$, with the observation as the result of the generation process defined as follows:

$$O(Rec_{i,t_{j+\Delta}}, Rec_{i,t_j}; \tau_{i,t_j}) = [g_{i,t_j}, \ell_{i,t_j}, t_{elapsed}]$$

The eventual observation is encapsulated into a Observation Message and sent to the Window Decision Module for storage. The consecutive process triggered by packet received event can be seen in Algorithm 1.

C. Window Decision Module

Window Decision Module consists of a Reinforcement Learning model to output a scaling factor to underlying TCP

socket periodically triggered by Action Request from Window Enforcement Module. An enforced congestion window is then calculated with the factor by the underlying TCP socket and included in ACK to the sender. The sender will overwrite its *cwnd* by the enforced congestion window. Therefore, to implement a continuous control to modified TCP, we choose Deep Deterministic Policy Gradient (DDPG) as agent in our design.

1) *State Generation*: An observation is generated on the packet arrival event while the action is requested periodically. In our design, the action period is set to long enough to ensure we have multiple observations stored by an Observation Repository. Therefore, we first use an exponential weighted moving average (EWMA) filter to smooth the observations before storing them in the repository. This is done to mitigate the noise when the observation is generated in a varying network environment e.g., Wi-Fi. After storing, when an action request is received, the average of the stored observations during this action period is calculated as input to our agent, seen as follows when n observations are stored:

$$s_t = \left[\frac{1}{n} \sum_{i=1}^n g_i, \frac{1}{n} \sum_{i=1}^n t_{elapsed,i} \right] \quad (7)$$

2) *Reward Design*: For improving network performance for each flow, the first component of our reward is based on network power metric to reflect individual flow performance, defined as follows:

$$r_{1,t} = power = \frac{\frac{1}{n} \sum_{i=1}^n g_i}{\frac{1}{n} \sum_{i=1}^n t_{elapsed,i}}$$

We also take the advantage of the global performance store in repository to design the second component for improving fairness among flows. Jain's fairness index is introduced considering N flows and $\bar{g}_i = \frac{1}{n} \sum_{i=1}^n g_{i,t}$ defined as follows:

$$r_{2,t} = J(\bar{g}) = \frac{\left(\sum_{i=1}^N \bar{g}_i \right)^2}{N \sum_{i=1}^N \bar{g}_i^2}$$

Eventually, we design our reward as:

$$r_t = \begin{cases} -2, & \text{if } \ell_t > 0 \\ r_{1,t} - (1 - r_{2,t})^2, & \text{otherwise} \end{cases} \quad (8)$$

3) *Training*: Our agent consists of an actor network determining a deterministic action $a = \mu(s; \theta^\mu)$ parameterised by θ^μ and a critic network to approximate the expected future return, defined as $Q(s, a; \theta^Q) = \mathbb{E}[R_t | s_t, a_t]$ with parameters θ^Q . The return R_t is the sum of discounted future return, i.e., $R_t = \sum_{i=t}^T \gamma^{i-t} r(s_i, a_i)$. Approaching to the approximation is done by updating the critic network through minimising the mean square error between its output and a target function (y_t), shown as follows:

$$L(\theta^Q) = \mathbb{E}[(Q(s_t, a_t; \theta^Q) - y_t)^2]$$

$$y_t = r(s_t, a_t) + \gamma Q^-(s_{t+1}, \mu^-(s_{t+1}; \theta^{\mu^-}); \theta^{Q^-})$$

where $Q^-(s, a; \theta^-)$ and $\mu^-(s; \theta^{\mu^-})$ are target critic network and target policy network. Their parameters are updated

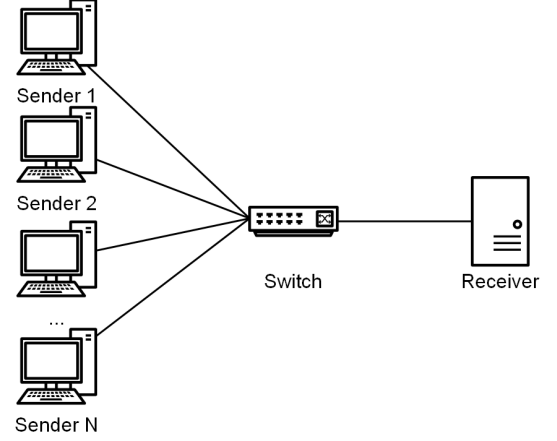


Fig. 4: A multi-to-one flow scenario.

by calculating the exponentially weighted moving average (EWMA) of critic network and policy network, i.e., $\theta^- \leftarrow \tau \times \theta + (1 - \tau) \times \theta^-$ where the τ is the weight parameter. Subsequently, with the updated critic network, policy performance of current actor network is predicted, expressed as $J(\theta^\mu) = \mathbb{E}[Q(s_t, \mu(s_t; \theta^\mu))]$. Training an actor network is done by updating its parameters through policy performance gradient ascent. Specifically, by applying the chain rule, the gradient of the policy performance, is computed as:

$$\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_{a_t} Q(s_t, \mu(s_t; \theta^\mu)) \nabla_{\theta^\mu} \mu(s_t; \theta^\mu)]$$

D. Window Enforcement Module

For an RL-controlled CC, it is important that waiting agent for action generation should not block the TCP normal behaviour. Therefore, we decouple the Window Decision Module, Receiver Measurement Module and standard feature to ensure none of them will block any other. Therefore, an ACK response will not be delayed in our scheme. In our design, an action request message will be sent to agent periodically by Window Decision Module. The resulting action is a factor within $[0, 1]$. Once the Window Decision Module receive the action, it will extract the latest *cwnd* in Receiver Measurement Module and compute the enforced congestion window ($cwnd_\mu$) using the following:

$$cwnd_\mu = cwnd \times scaling_factor^a, \quad n \in [0, 1] \quad (9)$$

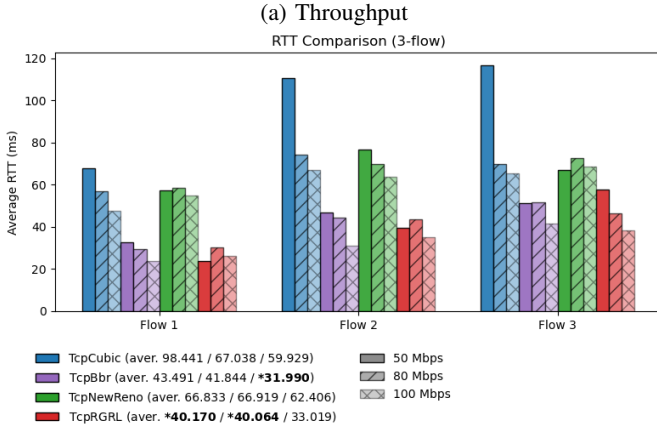
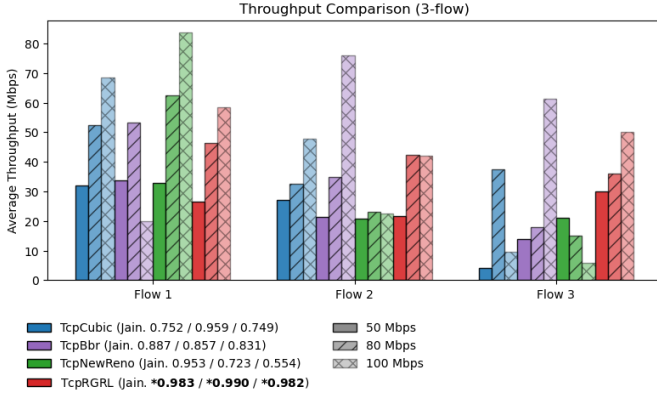
Then $cwnd_\mu$ will be included into the ACK header and take effect at the sender side.

V. SIMULATION SETUP AND RESULTS

Our scheme is implemented in ns-3.39 simulator. We set *scaling_factor* in Equation. 9 to 2. With the motivation to improve the performance of multiple flows to one receiver, we set up our topology as shown in Fig. 4. We compare our scheme with existing CC algorithms, i.e., NewReno, Cubic, and BBR.



Fig. 5: Cumulative reward per episode.



(b) RTT

Fig. 6: Performance comparison under different bottleneck bandwidths.

A. 3-flow scenario

In this scenario, we started three flows with a 10s interval. In particular, the three flows are active respectively in $[0 - 50s]$, $[10 - 40s]$, and $[20 - 30s]$, in which case we focus on the fairness performance of our scheme. We set the bandwidth in our topology respectively as 50 Mbps, 80 Mbps, and 100 Mbps. The base RTT is set to 20 ms in all simulations.

We trained our agent for 100 episodes where each episode is a complete simulation in ns-3.39. The reward per episode can be seen in Fig. 5. RGRL can converge its cumulative reward to a certain level different depending on the simulation setting.

The difference of reward convergence is due to the bandwidth and base RTT determining the reward distribution for agent since we include power in reward design.

The average throughput and fairness performance of our scheme can be seen in Fig. 6a. For the fairness we calculated Jain Index for the three flows in their active period. We can see that our scheme RGRL can have a better fairness than other CC algorithms in all three bandwidth settings, i.e., around 0.98. It is observed that BBR can achieve 0.88 in 50 Mbps case, which is decreased slightly when the bandwidth is lower, particularly, to 0.83 for 100 Mbps setting. In terms of the real-time throughput, as shown in Fig. 9, BBR flows behave that the later flow dominates the bandwidth and the flow 1 can hardly recover to fully utilise the bandwidth after 10 seconds. BBR flow 1 eventually gets 20.08 Mbps average throughput much lower than other two flows which is not clearly reflected in Jain Index.

NewReno Jain index is decreased significantly when the bandwidth is higher, from 0.88 in 50 Mbps to 0.5536 in 100 Mbps. For the real-time throughput in Fig. 7, NewReno behaves a rapid convergence. Cubic achieves a good performance, 0.9592 only in 80 Mbps setting while remains lower than 0.80 in other cases. The real-time throughput can be seen in Fig. 8. RGRL has an advantage in convergence that is more significant when the bandwidth is higher.

RGRL performs better in convergence when bandwidth is higher due to the coarse-grained control from our periodic control design. Particularly, a higher bandwidth results in a larger BDP, which requires a longer time for the *cwnd* to increase in AIMD and provides sufficient time before overutilising for RGRL to decrease *cwnd* to a fairer level.

We also present the average RTT of flow for each CC algorithm that can be seen in Fig. 6b. Despite Cubic and NewReno achieves satisfying fairness performance in specific setting, they are out of the cost of RTT from recklessly increasing *cwnd*, i.e., over 60 ms for NewReno and at least 59.93 for Cubic among the cases. The latter can achieve even 98.44 ms in 50 Mbps setting. RGRL can achieve a lower average RTT compared with BBR in 50 Mbps and 80 Mbps. For the 100 Mbps, our scheme can achieve preferable performance to 33.02 ms similar to BBR's 31.99 ms whilst BBR suffers convergence problem shown in real-time throughput comparison in Fig. 9.

B. Periodic traffic scenario

We also tested random flows for simulating IoT traffic pattern using periodic traffic scenario. Therefore, all IoT nodes are set to send messages with the same period. Their first messages are initialised following the uniform distribution within the period. In our setting, the sending period is set to 0.5s. The messages are set to 1 MB delivered as packets sized of 1448 bytes. We compared our scheme performance with other CC algorithm in two scenarios, i.e., (i) 50 IoT nodes with 100 Mbps bandwidth wired connection using topology shown as Fig. 4, and (ii) 30 IoT nodes using WiFi. Both scenarios last for 50 seconds.

In scenario (i), we compare the flow completion time (FCT) shown as in Fig. 10 where the best performances are

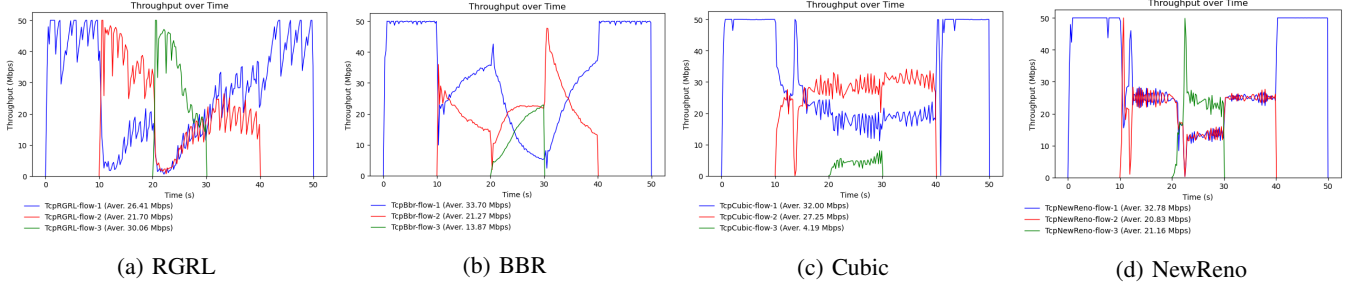


Fig. 7: Throughput and fairness performance in 3-flow scenario with bandwidth set to 50 Mbps.

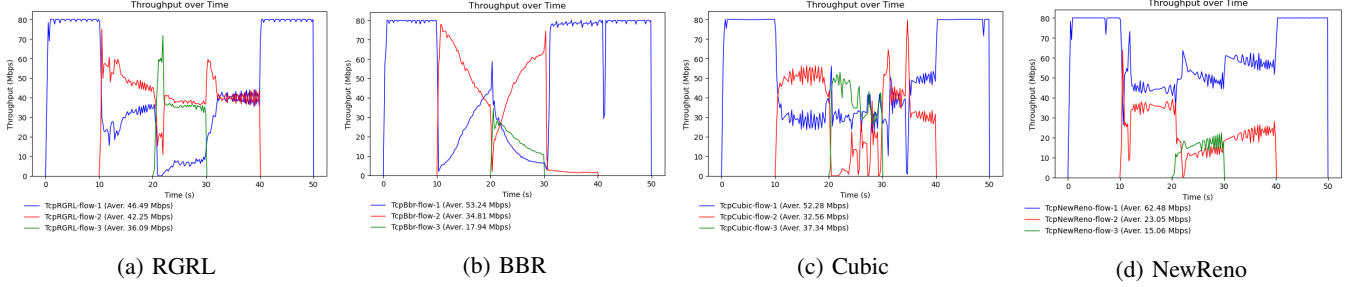


Fig. 8: Throughput and fairness performance in 3-flow scenario with bandwidth set to 80 Mbps.

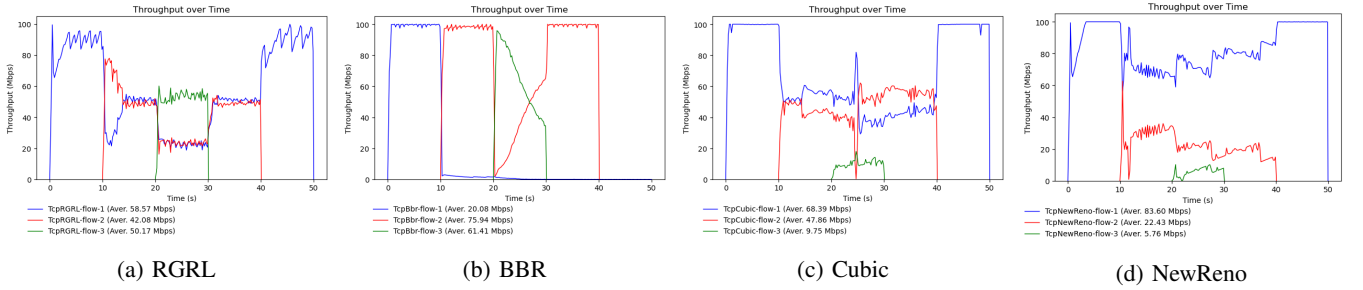


Fig. 9: Throughput and fairness performance in 3-flow scenario with bandwidth set to 100 Mbps.

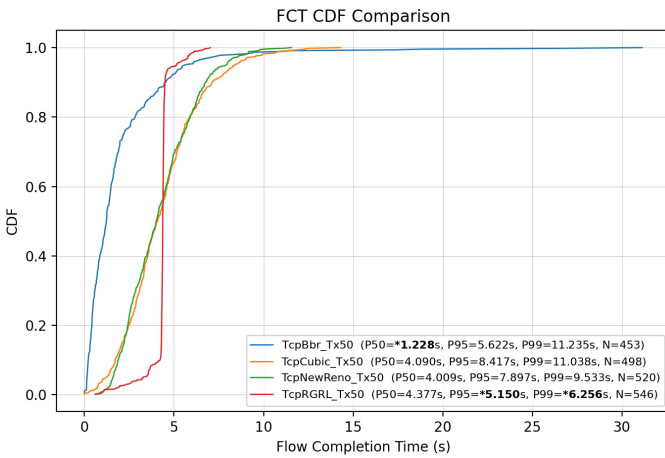


Fig. 10: Flow completion time comparison of 50 IoT nodes using wired connection.

highlighted. Our scheme has the lowest FCT at P90 and P95 reducing the tail latency. In terms of P50 performance, our scheme RGRL achieves to the level close to Cubic and

NewReno which are mainly used in network today. Despite other CC algorithms have lower FCT at P50, they are out of cost of more packet losses hurting the communication reliability. The packet losses of each scheme can be seen in Fig. 11. Our scheme has the least packets losses with the lowest tail latency.

In scenario (ii), the FCT is shown in Fig. 12. RGRL still performs the lowest FCT at P95 and P99, 3.803s and 5.056s. And RGRL has P50 performance, 2.356s better than Cubic and NewReno but slightly higher than BBR, 1.719s. A difference from scenario (i) is observed that BBR achieves much worse performance at P95, 8.155s much higher than other CCs that achieve a value around 4s. BBR also has the extreme high FCT at P99, 14.270s compared to other CCs that achieve around 5s. BBR can not guarantee a sufficient low tail latency in periodic traffics scenario. Also, it is observed that RGRL has the least packet loss compared to other CCs as shown in Fig. 13. BBR suffers significant packet losses compared to other CCs in this scenario which indicates more retransmissions for IoT devices that exhausting the energy consumption.

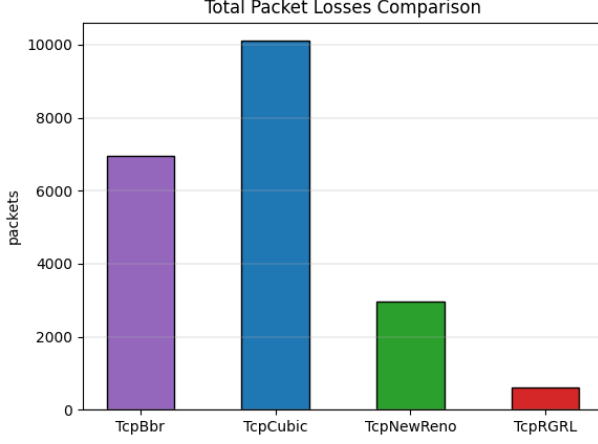


Fig. 11: Packet losses comparison of 50 IoT nodes using wired connection.

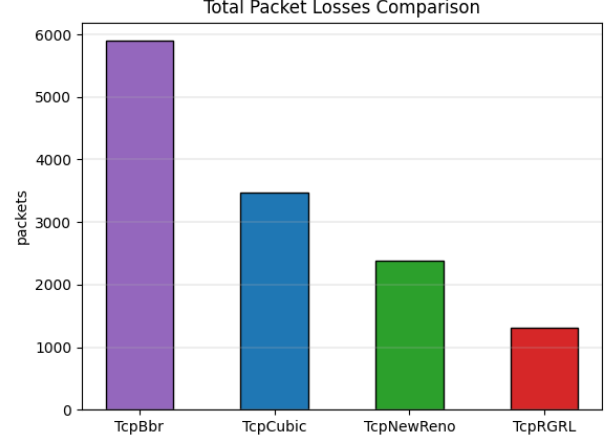


Fig. 13: Packet losses comparison of 30 IoT nodes using WiFi.

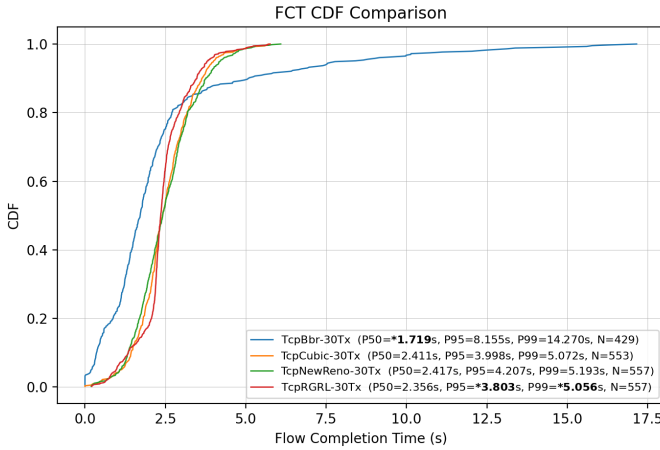


Fig. 12: Flow completion time comparison of 30 IoT nodes using WiFi.

C. Mixed traffic scenario

We then test our scheme considering event-based application traffics mixed with periodic traffics. To simulate mixed traffics, we set 10 nodes to send message with an interval time sampling according to Equation. 2 with $\frac{1}{\lambda} = 1.125s$ and 40 nodes to generate periodic traffic every 0.5 seconds. Such that the message rate from event-based application account for 10% of the total message rate.

VI. CONCLUSION

The conclusion goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] V. K. Jain, A. P. Mazumdar, P. Faruki, and M. C. Govil, "Congestion control in internet of things: Classification, challenges, and future directions," *Sustainable Computing: Informatics and Systems*, vol. 35, p. 100678, 2022.
- [2] P. Anitha, H. Vimala, and J. Shreyas, "Comprehensive review on congestion detection, alleviation, and control for iot networks," *Journal of Network and Computer Applications*, vol. 221, p. 103749, 2024.
- [3] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, D. Niyato, O. Dobre, and H. V. Poor, "6g internet of things: A comprehensive survey," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 359–383, 2021.
- [4] A. Betzler, C. Gomez, I. Demirkol, and J. Paradells, "Coap congestion control for the internet of things," *IEEE Communications Magazine*, vol. 54, no. 7, pp. 154–160, 2016.
- [5] R. S. Mogensen, I. Rodriguez, G. Berardinelli, G. Pocovi, and T. Kolding, "Empirical iiot data traffic analysis and comparison to 3gpp 5g models," in *2021 IEEE 94th Vehicular Technology Conference (VTC2021-Fall)*. IEEE, 2021, pp. 1–7.
- [6] T. Raunholt, I. Rodriguez, P. Mogensen, and M. Larsen, "Towards a 5g mobile edge cloud planner for autonomous mobile robots," in *2021 IEEE 94th Vehicular Technology Conference (VTC2021-Fall)*. IEEE, 2021, pp. 01–05.
- [7] C. Gomez, A. Arcia-Moret, and J. Crowcroft, "Tcp in the internet of things: From ostracism to prominence," *IEEE Internet Computing*, vol. 22, no. 1, pp. 29–41, 2018.
- [8] S. Kumar, M. P. Andersen, H.-S. Kim, and D. E. Culler, "Performant {TCP} for {Low-Power} wireless networks," in *17th USENIX symposium on networked systems design and implementation (NSDI 20)*, 2020, pp. 911–932.
- [9] L. Mhamdi and H. Abdul Khalek, "Congestion control in constrained internet of things networks," *IET Wireless Sensor Systems*, vol. 13, no. 6, pp. 247–255, 2023.
- [10] S. Floyd, T. Henderson, and A. Gurtov, "The newreno modification to tcp's fast recovery algorithm," Tech. Rep., 2004.
- [11] S. Ha, I. Rhee, and L. Xu, "Cubic: a new tcp-friendly high-speed tcp variant," *ACM SIGOPS operating systems review*, vol. 42, no. 5, pp. 64–74, 2008.
- [12] L. S. Brakmo, S. W. O'malley, and L. L. Peterson, "Tcp vegas: New techniques for congestion detection and avoidance," in *Proceedings of the conference on Communications architectures, protocols and applications*, 1994, pp. 24–35.
- [13] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "Bbr: Congestion-based congestion control," *Communications of the ACM*, vol. 60, no. 2, pp. 58–66, 2017.
- [14] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center tcp (dctcp)," in *Proceedings of the ACM SIGCOMM 2010 Conference*, 2010, pp. 63–74.
- [15] J. Zhang, F. Ren, X. Yue, R. Shu, and C. Lin, "Sharing bandwidth by allocating switch buffer in data center networks," *IEEE Journal on Selected Areas in Communications*, vol. 32, no. 1, pp. 39–51, 2013.
- [16] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "Qtcp: Adaptive congestion control with reinforcement learning," *IEEE Transactions on Network Science and Engineering*, vol. 6, no. 3, pp. 445–458, 2018.
- [17] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on internet congestion control," in

- International conference on machine learning*. PMLR, 2019, pp. 3050–3059.
- [18] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, “{PCC} vivace: {Online-Learning} congestion control,” in *15th USENIX symposium on networked systems design and implementation (NSDI 18)*, 2018, pp. 343–356.
 - [19] S. Abbasloo, C.-Y. Yen, and H. J. Chao, “Classic meets modern: A pragmatic learning-based congestion control for the internet,” in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 632–647.
 - [20] L. Giacomoni and G. Parisi, “Reinforcement learning-based congestion control: A systematic evaluation of fairness, efficiency and responsiveness,” in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 1451–1460.
 - [21] F. Metzger, T. Hoßfeld, A. Bauer, S. Kounev, and P. E. Heegaard, “Modeling of aggregated iot traffic and its application to an iot cloud,” *Proceedings of the IEEE*, vol. 107, no. 4, pp. 679–694, 2019.
 - [22] X. Nie, Y. Zhao, Z. Li, G. Chen, K. Sui, J. Zhang, Z. Ye, and D. Pei, “Dynamic tcp initial windows and congestion control schemes through reinforcement learning,” *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1231–1247, 2019.
 - [23] H. Tian, X. Liao, C. Zeng, J. Zhang, and K. Chen, “Spine: An efficient drl-based congestion control with ultra-low overhead,” in *Proceedings of the 18th international conference on emerging networking experiments and technologies*, 2022, pp. 261–275.
 - [24] C.-Y. Yen, S. Abbasloo, and H. J. Chao, “Computers can learn from the heuristic designs and master internet congestion control,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 255–274.
 - [25] J. Navarro-Ortiz, P. Romero-Diaz, S. Sendra, P. Ameigeiras, J. J. Ramos-Munoz, and J. M. Lopez-Soler, “A survey on 5g usage scenarios and traffic models,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 905–929, 2020.
 - [26] 3GPP, “Traffic model for m2m services,” TSG-RAN WG2 Meeting #68-bis, document R2-100204, Sophia Antipolis, France, Jan. 2010. [Online]. Available: http://www.3gpp.org/ftp/tsg_ran/WG2_RL2/TSGR2_68bis/docs/R2-100204.zip